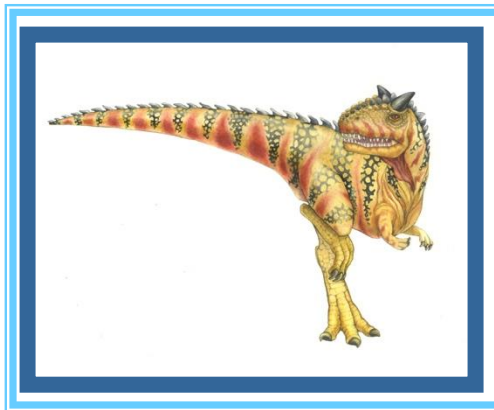


Chapter 5: CPU Scheduling





Chapter 5: CPU Scheduling

- ❑ Basic Concepts
- ❑ Scheduling Criteria
- ❑ Scheduling Algorithms
- ❑ Thread Scheduling
- ❑ Multiple-Processor Scheduling
- ❑ Real-time Scheduling
- ❑ Operating Systems Examples
- ❑ Algorithm Evaluation





Objectives

- To introduce CPU scheduling, which is the basis for **multiprogrammed** operating systems
- To describe various CPU-scheduling algorithms
- To discuss evaluation **criteria** for selecting a CPU-scheduling algorithm for a particular system





CPU Scheduling

- ❑ **Maximum CPU utilization** obtained with multiprogramming
 - ❑ As multiprogram running at the same time, they compete for resources, so scheduling is necessary, especially CPU.
- ❑ CPU Scheduling is a fundamental OS function
 - ❑ Every time a process has to wait, another process can take over use of CPU.





Basic Concepts

- CPU-I/O Burst Cycle
- CPU Scheduler
- CPU Scheduling Scheme
- CPU Dispatcher





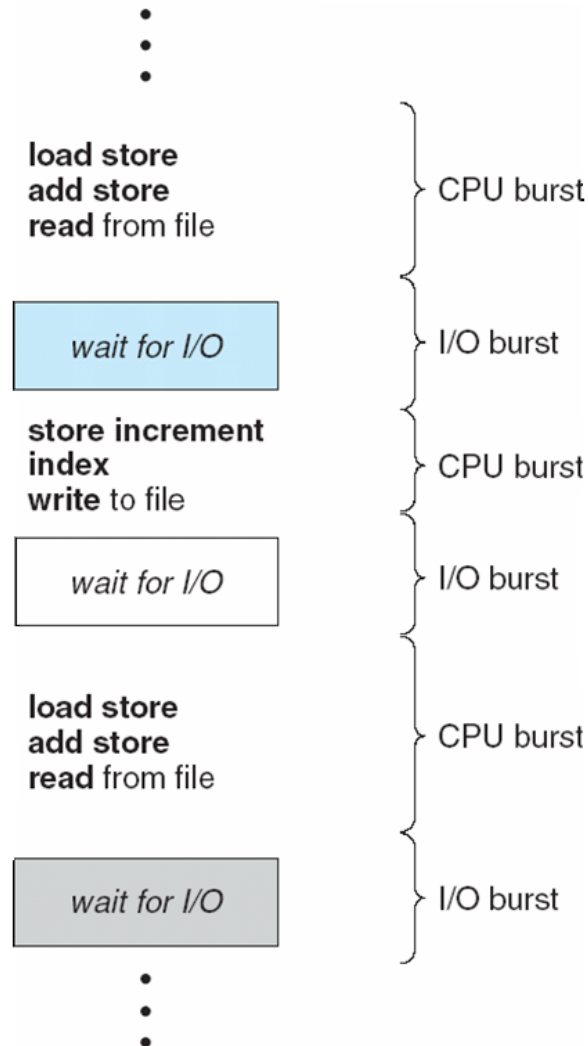
CPU-I/O Burst Cycle

- **CPU-I/O Burst Cycle** – Process execution consists of a cycle of CPU execution and I/O wait.
- CPU burst distribution
 - Usually, a large number of short CPU bursts and a small number of long CPU bursts
 - An I/O-bound program typically has many short CPU bursts
 - A CPU-bound program might have a few long CPU bursts.



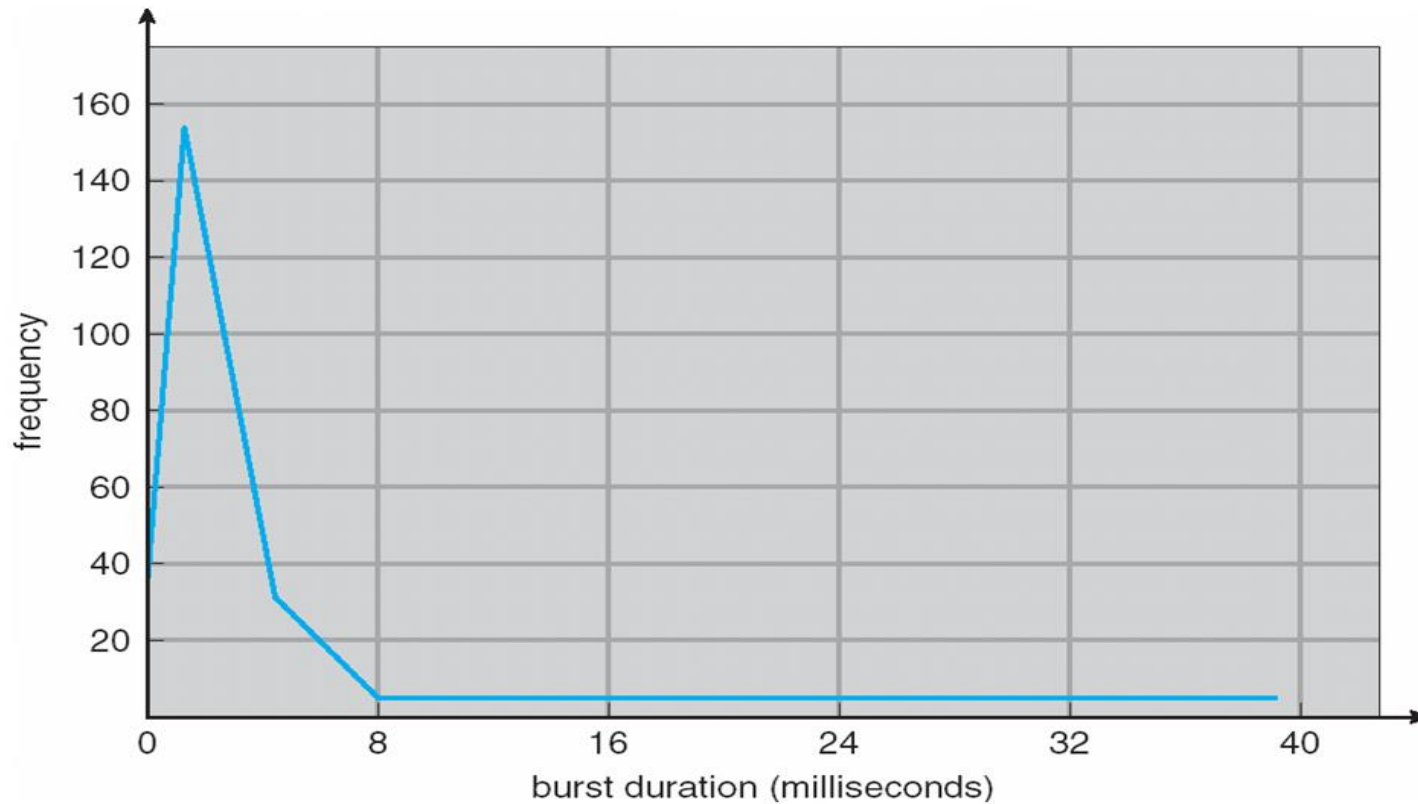


Alternating Sequence of CPU And I/O Bursts





Histogram of CPU-burst Times





CPU Scheduler

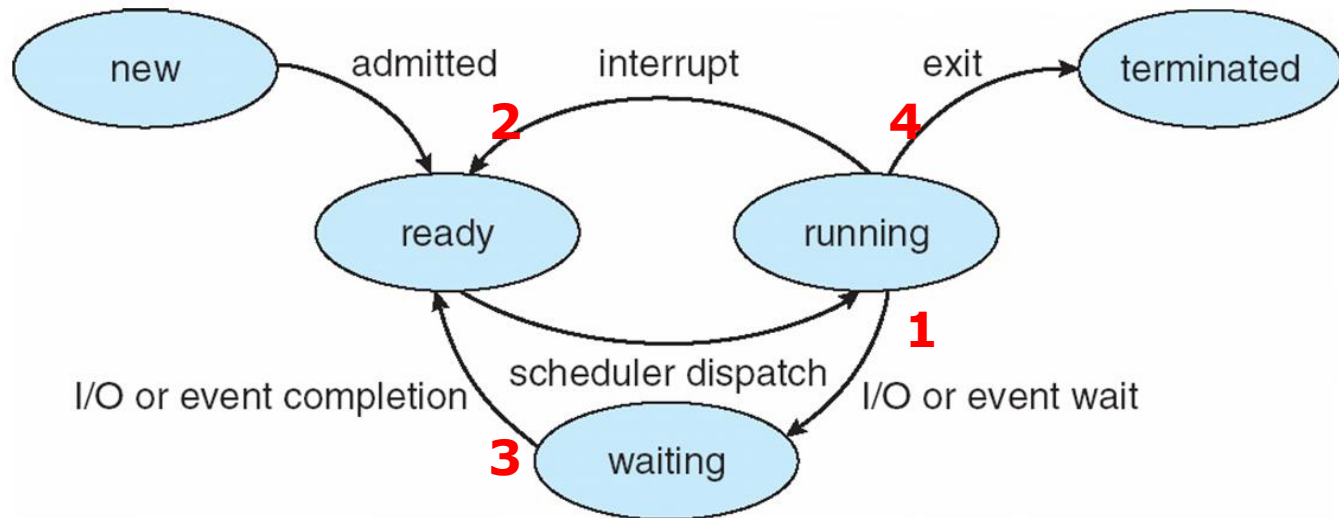
- ❑ **Selects** from among the processes in memory that are **ready** to execute, and allocates the CPU to one of them;
- ❑ Many processes in “ready” state;
- ❑ Which ready process to pick to run on the CPU?





CPU Scheduler

- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates



- Scheduling under 1 and 4 is **nonpreemptive**
- All other scheduling is **preemptive**





CPU Scheduling scheme

- Nonpreemptive Scheduling
 - Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
- Preemptive scheduling
 - According to some rule, the CPU utilization of a process can be preempted .
 - ▶ Timer
 - ▶ Priority





Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to **stop one process** and **start another running**
 - overhead





Scheduling Criteria

- ❑ **CPU utilization** – keep the CPU as busy as possible
- ❑ **Throughput** – # of processes that complete their execution per time unit
- ❑ **Turnaround time** – amount of time to execute a particular process
- ❑ **Waiting time** – amount of time a process has been waiting in the ready queue
- ❑ **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)





Scheduling Algorithm Optimization Criteria

- ❑ Max CPU utilization
- ❑ Max throughput
- ❑ Min turnaround time
- ❑ Min waiting time
- ❑ Min response time
- ❑ Fairness: everyone makes progress, no one starves

“The perfect CPU scheduler”





There Two processes in One CPU

- Process A needs to run one hour
- Process B needs to run one minute

Which one should be run first by OS?





Scheduling Algorithm

- FCFS
- SJF
- Priority scheduling
- Round Robin (RR)
- Multilevel Queue
- Multilevel Feedback Queue





First-Come, First-Served (FCFS) Scheduling

- ❑ FCFS is the simplest CPU scheduling algorithm
- ❑ The process that requests the CPU first is allocated the CPU first
- ❑ Implementation:
 - ❑ Manage with a **FIFO queue**
- ❑ The average waiting time under the FCFS policy is often quite long.

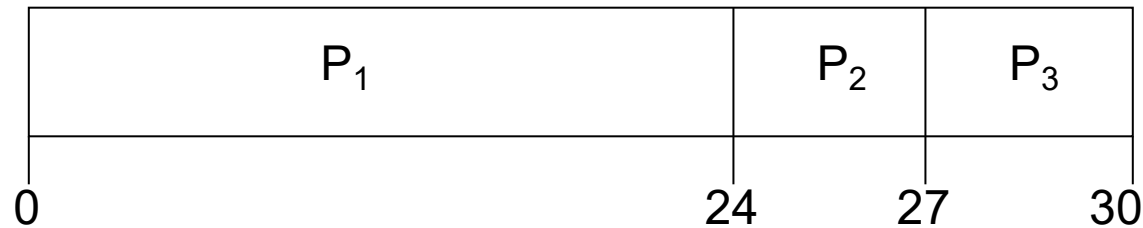




First-Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$





FCFS Scheduling (Cont)

Suppose that the processes arrive in the order

$$P_2, P_3, P_1$$

□ The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case





Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives **minimum average waiting time** for a given set of processes.
 - The difficulty is knowing the length of the next CPU request





Shortest-Job-First (SJF) Scheduling

- Two schemes:
 - **nonpreemptive** – once CPU given to the process it can not be preempted until completes its CPU burst
 - **Preemptive** – if a new process arrives with CPU burst length less than remaining time of currently executing process, preempt. This scheme is known as the Shortest-Remaining-Time-First (SRTF).

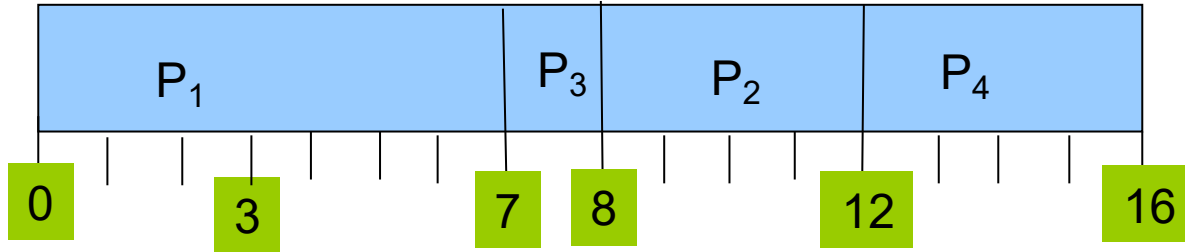




Example of Non-Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

□ SJF (non-preemptive)



□ Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

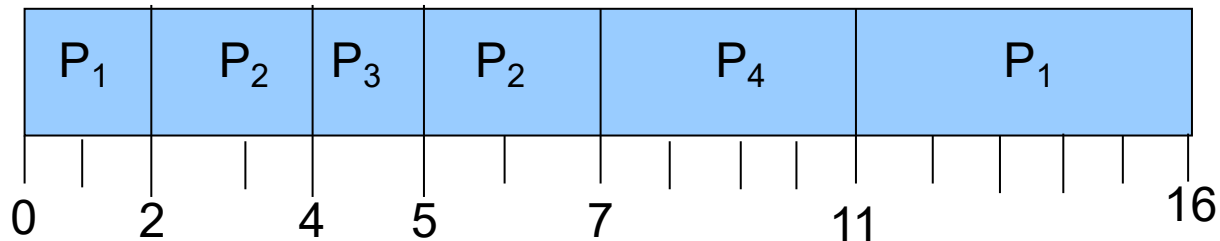




Example of Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

□ SJF (preemptive)



□ Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$





Determining Length of Next CPU Burst

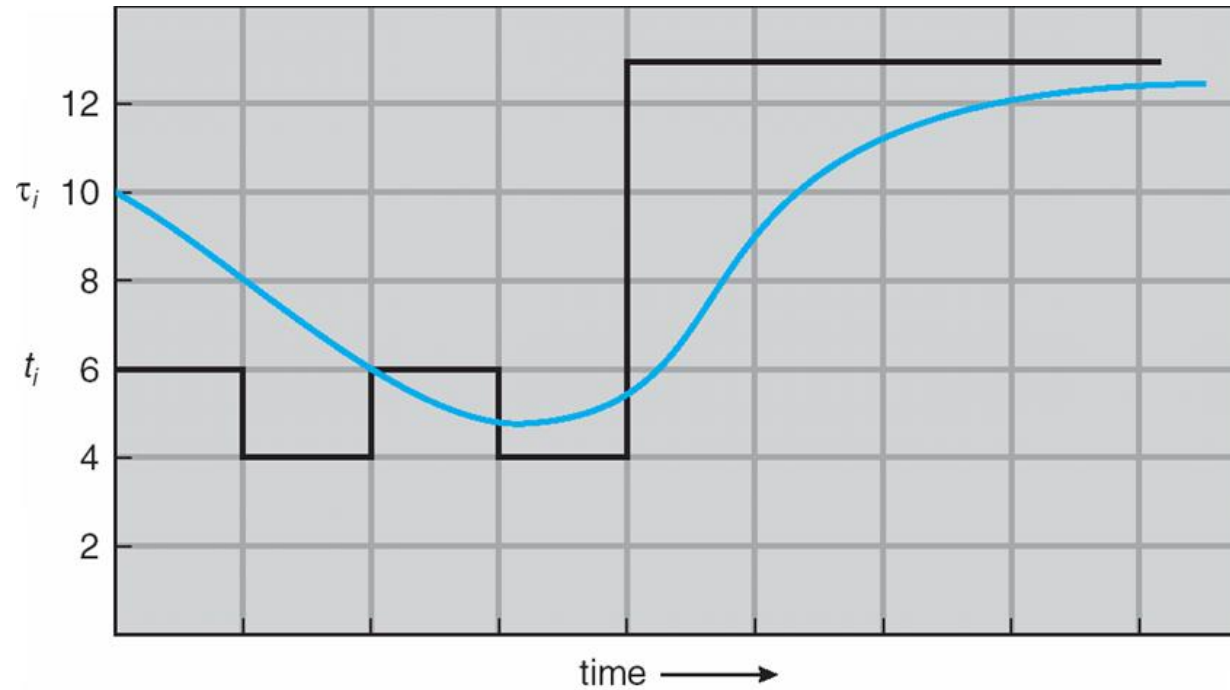
- Can only estimate the length (CPU BURST)
- Can be done by using the length of previous CPU bursts, using exponential averaging

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \leq \alpha \leq 1$
4. Define : $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.





Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...





Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - nonpreemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time





Priority Scheduling

- Problem $\equiv$ **Starvation** – low priority processes may never execute
- An interesting example:
 - At MIT, they found that a low-priority process submitted in 1967 had not yet been run when they shut down the IBM 7094 in 1973.
- Solution $\equiv$ **Aging** – increase the priority of the process waiting for a long time.





Priority Scheduling

- OS Solutions to process starvation
 - Decreasing priority & aging: the Unix approach
 - ▶ Decrease priority of **CPU-intense process**
 - ▶ Exponential averaging of CPU usage to slowly **increase priority of blocked process**
 - ▶ $p_pri = \min\{127, (p_cpu/16 + PUSER + p_nice)\}$,
 - Priority Elevation: the Windows approach
 - ▶ Increase priority of **a thread on I/O completion**
 - ▶ System gives **starved threads** an extra burst





Round Robin (RR)

- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is **preempted** and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process **waits** more than $(n-1)q$ time units.
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

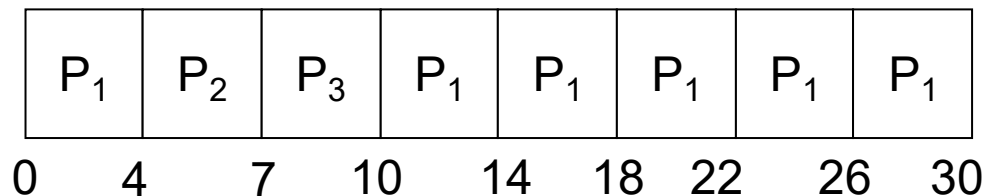




Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

□ The Gantt chart is:

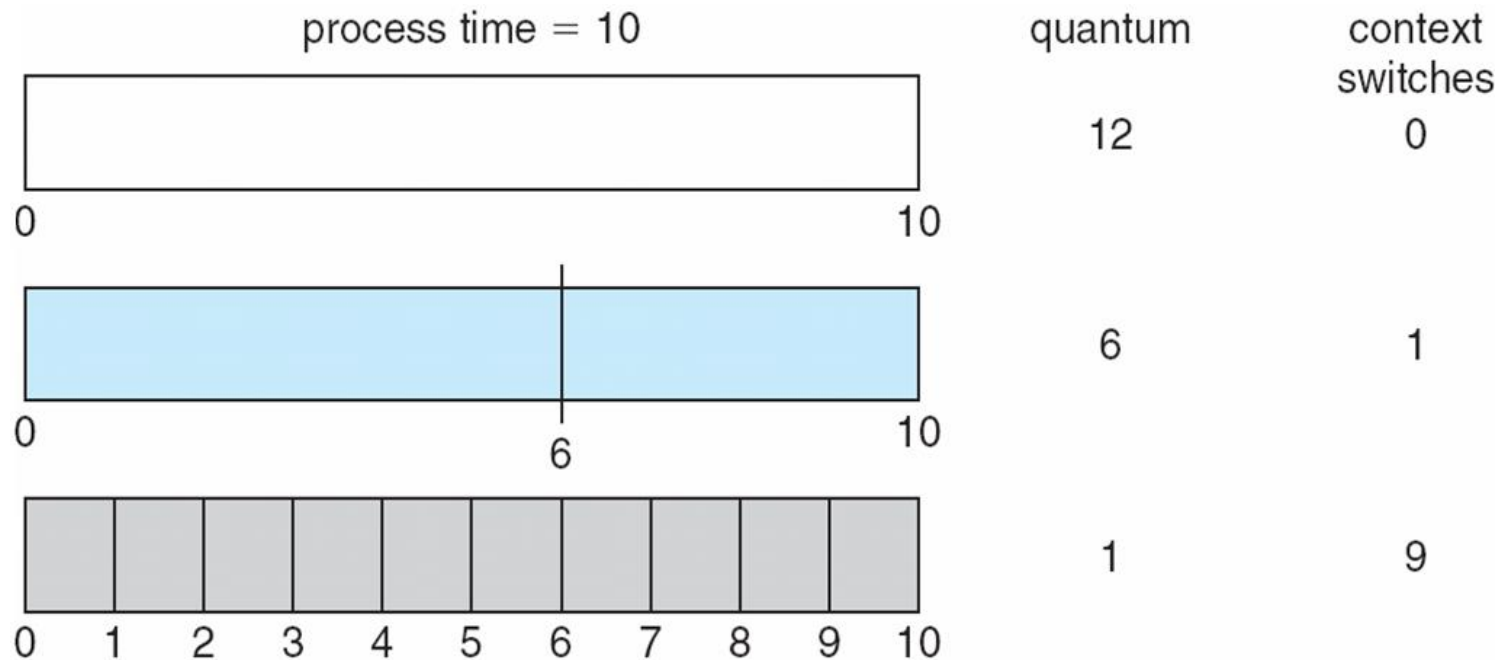


□ Typically, higher average turnaround than SJF, but better *response*



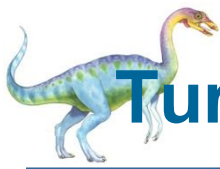


Time Quantum and Context Switch Time

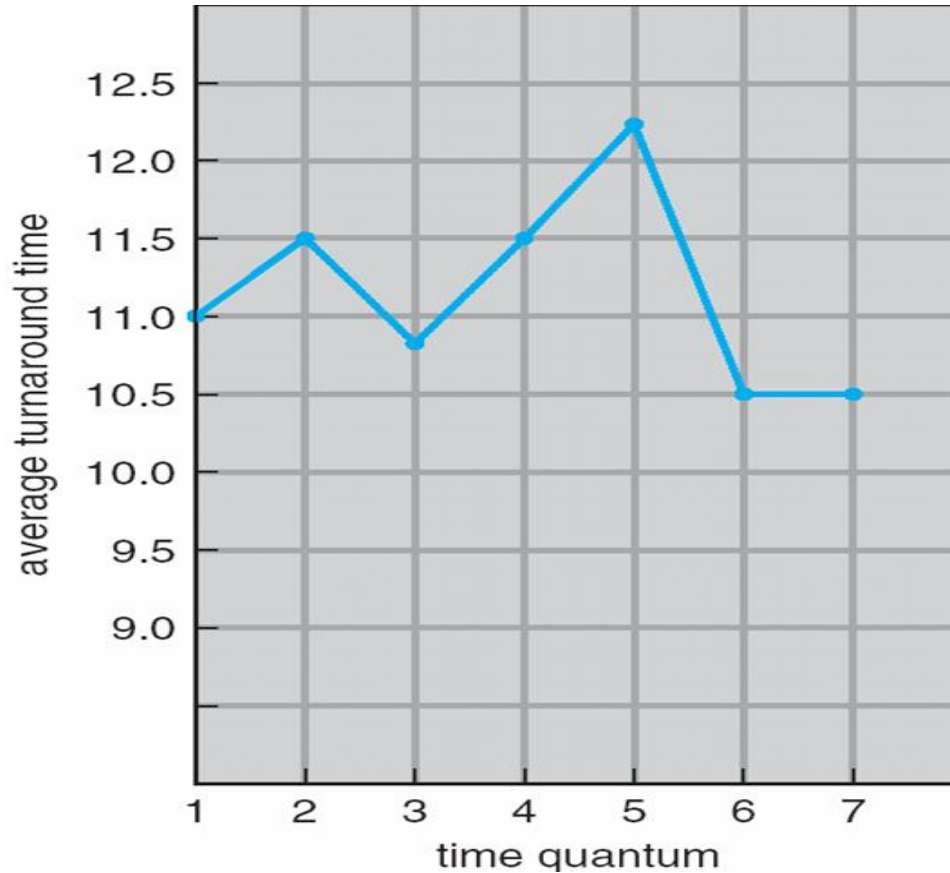


- Longer quantum yields shorter average turnaround time?





Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7





Time Quantum and Context Switch Time

- Performance depends on **length of the timeslice**
 - Context switching isn't a free operation.
 - If timeslice time is set too high
 - ▶ attempting to amortize context switch cost, you get FCFS.
 - If it's set too low
 - ▶ you're spending all of your time context switching between threads.
- Timeslice frequently set to **10~100 milliseconds**
- Context switches typically **cost < 1 millisecond**





Multilevel Queue

- Ready queue is partitioned into separate queues:
e.g. foreground (interactive)
background (batch)
- Each queue has its own scheduling algorithm
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues
 - **Fixed** priority scheduling; (i.e., serve all from foreground then from background). Possibility of **starvation**.
 - **Time slice** – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR, 20% to background in FCFS





Question 1

- 现有三个同时到达的作业J1、J2和J3，它们的执行时间分别是 $T1$ 、 $T2$ 和 $T3$ ，并且 $T1 < T2 < T3$ 。系统按单道方式运行且采用短作业有限算法，则平均周转时间是_____？
 - A. $T1+T2+T3$
 - B. $(T1+T2+T3)/3$
 - C. $(3T1+2T2+T3)/3$
 - D. $(T1+2T2+3T3)/3$





Question 2

- 现有三个同时到达的作业J1、J2和J3，它们的执行时间分别是T1、T2和T3，并且 $T1 < T2 < T3$ 。系统按单道方式运行且采用短作业优先算法，则平均周转时间是_____？

作业	达到时间	运行时间	完成时间	周转时间
J1	0	T1	T1	T1
J2	0	T2	T1+T2	T1+T2
J3	0	T3	T1+T2+T3	T1+T2+T3
平均周转时间		$(3T1+2T2+T3)/3$		





Question 3

- 一作业8:00达到系统, 估计运行时间为1小时。若10:00开始执行该作业, 其响应比是_____。

- A. 2
- B. 1
- C. 3
- D. 0.5

✓ 解析:

响应比=响应时间/服务时间, 响应时间=等待时间+服务时间
=2+1=3, $R_p=3/1=3$ 。





Question 4

- 设有三个作业，它们的到达时间和运行时间如表所示，并在一台处理机上按单道方式运行。如按响应比高者优先算法，则作业执行的次序是_____。

作业	达到时间	运行时间
J1	8:00	2
J2	8:30	1
J3	9:30	0.25

➤ A. J1,J2,J3

➤ B. J1,J3,J2

➤ C. J2,J3,J1

➤ D. J3,J2,J1



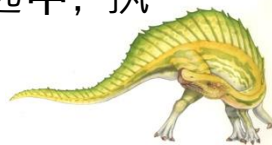


Question 4

- 设有三个作业，它们的到达时间和运行时间如表所示，并在一台处理机上按单道方式运行。如按响应比高者优先算法，则作业执行的次序是_____。

作业	达到时间	运行时间
J1	8:00	2
J2	8:30	1
J3	9:30	0.25

- ✓ ①开始时只有作业J1，作业J1被选中，执行时间两小时；②作业1执行完毕后，作业J2和J3都到达，响应比分别为 $(2-0.5)/1=1.5$ ， $(2-1.5)/0.25=2$ ，作业J3被选中，执行时间0.25小时；③作业J3执行完毕后，作业J2被选中，执行时间1小时，所有执行顺序为J1，J3，J2。





Question 5

有5个批处理作业（A、B、C、D、E）几乎同时到达一个计算中心，估计的运行时间分别为2，4，6，8、10分钟，它们的优先数分别为1，2，3，4，5（1为最低优先级），对下面的每种调度算法，分别计算作业的平均周转时间。

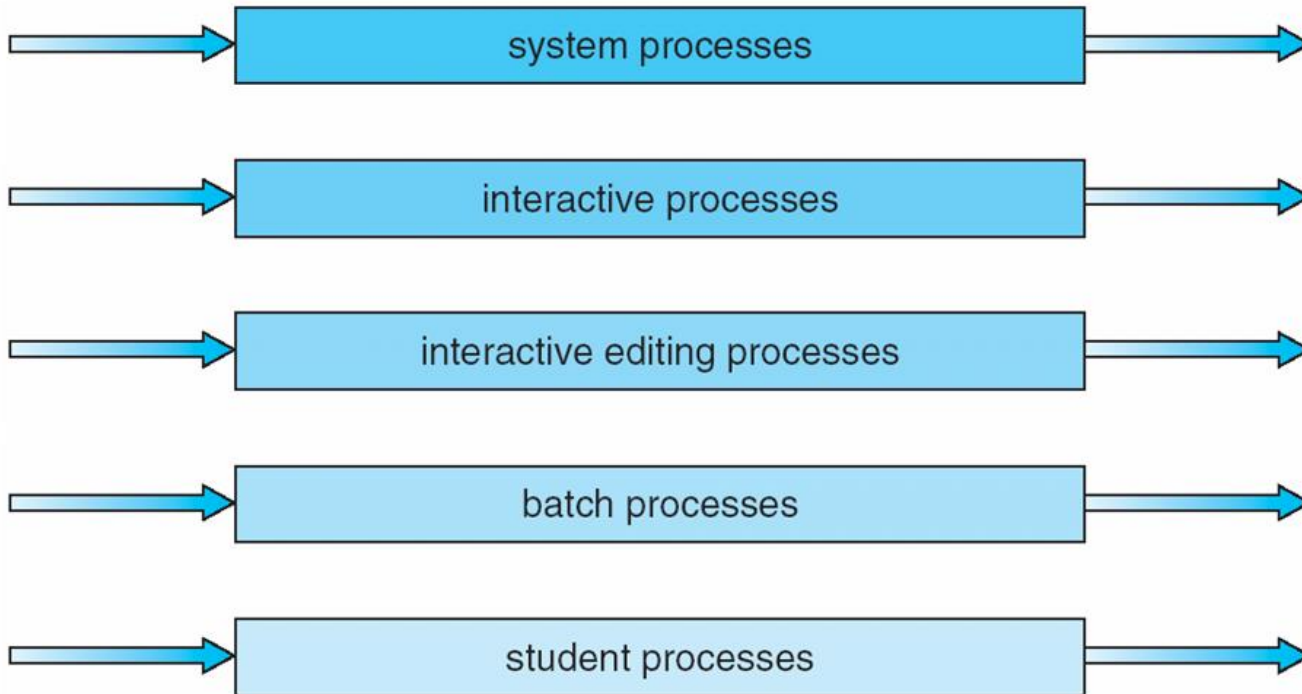
- 1 最高优先级先
- 2 时间片轮转（时间片为2分钟）
- 3 FIFO（作业到达顺序为C、D、B、E、A）
- 4 短作业优先





Multilevel Queue Scheduling

highest priority



lowest priority





Multilevel Feedback Queue

- A process can **move** between the various queues; aging can be implemented this way
- The idea is to separate processes with different CPU burst characteristics.
 - If a process uses **too much CPU** time, it will be moved to a lower priority queue.
 - If a process that **waits too long** in a lower priority queue may be moved to a higher priority queue.
- Avoiding **starvation**.





Multilevel Feedback Queue

- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of **queues**
 - **scheduling** algorithms for each queue
 - method used to determine when to **upgrade** a process
 - method used to determine when to **demote** a process
 - method used to determine **which queue** a process will **enter** when that process needs service

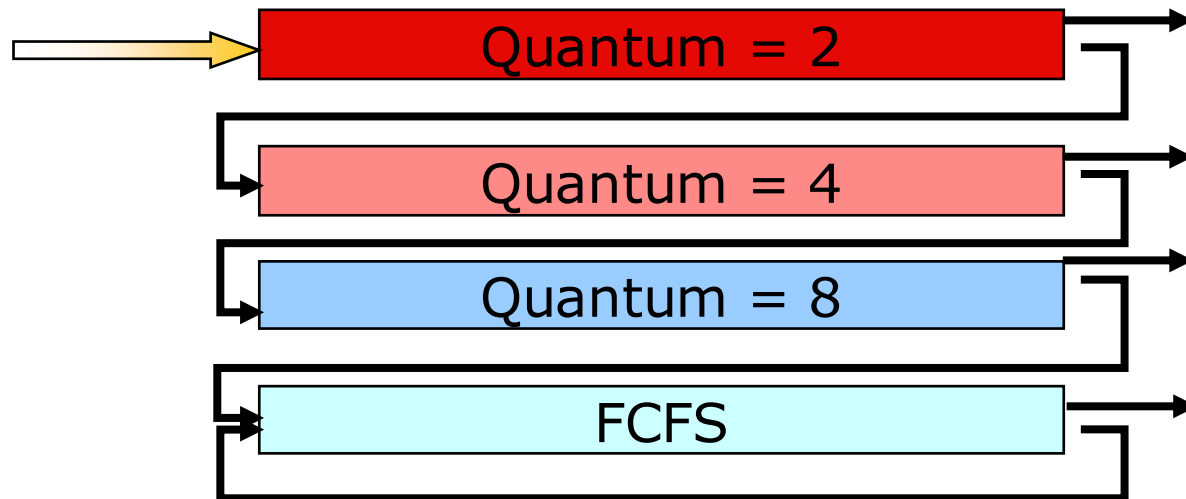




Example of Multilevel Feedback Queue

- Four queues:
 - Q_0 – RR with time quantum 2 milliseconds
 - Q_1 – RR time quantum 4 milliseconds
 - Q_2 – RR time quantum 8 milliseconds
 - Q_3 – FCFS

Highest priority

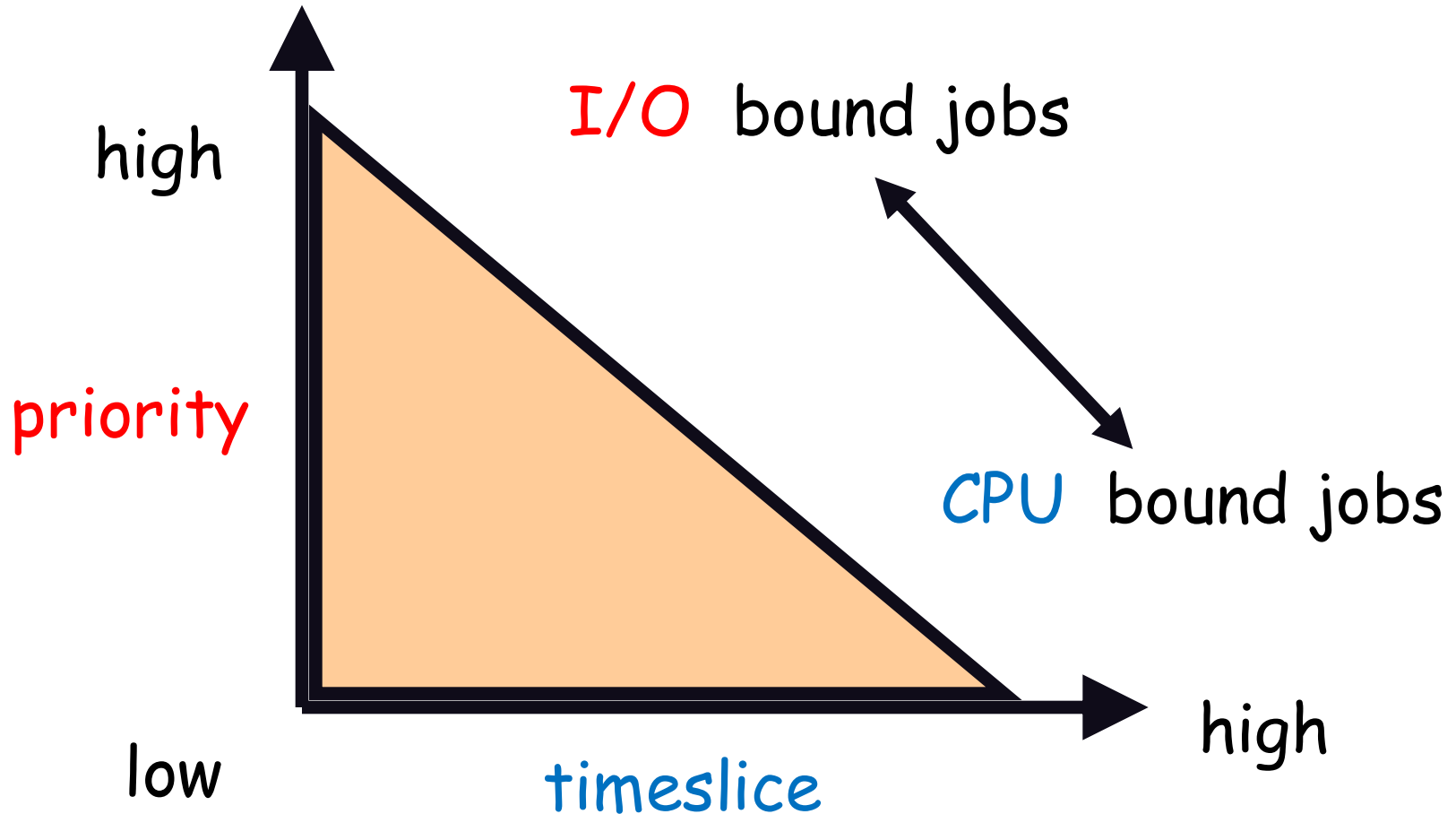


Lowest priority





A Multi-level System





Thread Scheduling

- Distinction between user-level and kernel-level threads
- Many-to-one and many-to-many models, **thread library schedules user-level threads to run on LWP**
 - Known as **process-contention scope (PCS)** since scheduling competition is **within the process**
- **Kernel thread scheduled onto available CPU is system-contention scope (SCS)** – competition among **all threads in system**
 - Systems using the one-to-one model (such as Windows XP, Solaris 9, Linux) schedule threads using only SCS





Pthread Scheduling

- API allows **specifying** either PCS or SCS during thread creation
 - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
 - ▶ Schedules user-level threads onto available LWPs
 - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling.
 - ▶ **Create and bind an LWP** for each user-level thread on many-to-many systems, Mapping threads using one-to-one policy





Pthread Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
    int i;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* set the scheduling algorithm to PROCESS or SYSTEM */
    pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
    /* set the scheduling policy - FIFO, RT, or OTHER */
    pthread_attr_setschedpolicy(&attr, SCHED_OTHER);
    /* create the threads */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);
}
```





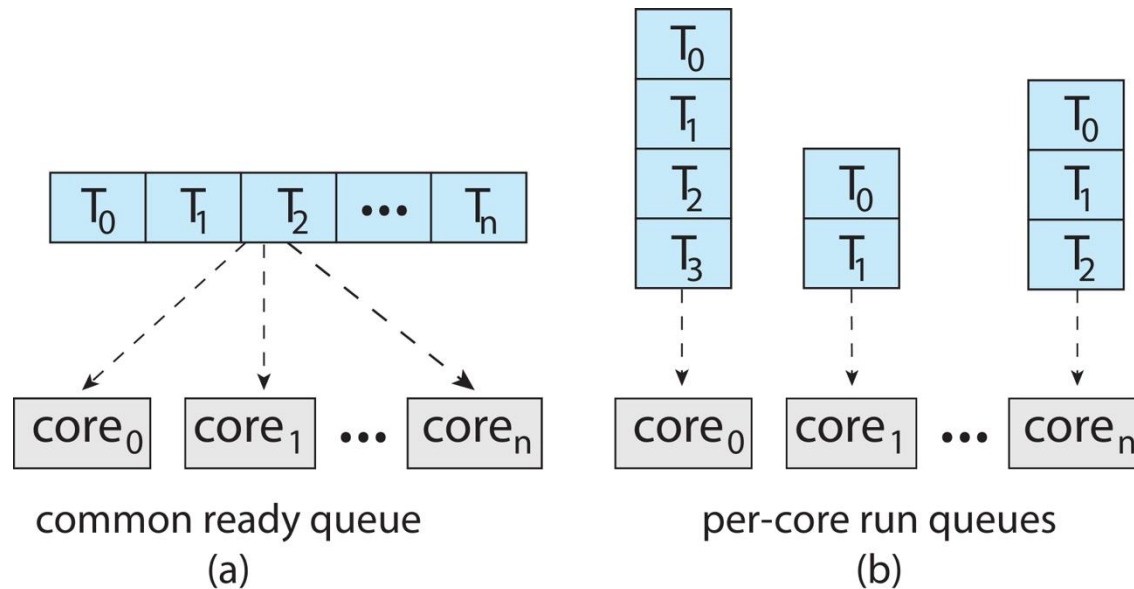
Multiple-Processor Scheduling

- ❑ CPU scheduling more complex when multiple CPUs are available
- ❑ **Homogeneous processors** within a multiprocessor
- ❑ **Asymmetric multiprocessing** – only one processor accesses the system data structures, **alleviating** the need for **data sharing**
- ❑ **Symmetric multiprocessing (SMP)** – each processor is self-scheduling
 - ❑ all processes in **common ready queue**
 - ❑ or each has its own **private queue** of ready processes
 - ❑ In **most modern OS** supporting SMP, each processor has a **private ready queue**.





Multiple-Processor Scheduling





Multiple-Processor Scheduling

- **Processor affinity** – process has affinity for processor on which it is currently running
 - **soft affinity**
 - ▶ OS has a policy of **attempting** to keep a process running on the **same processor** but not guaranteeing that it will do so
 - ▶ It is possible for a process to migrate between processors
 - **hard affinity**
 - ▶ OS provides system calls allowing a process to specify that it is **not to migrate** to other processors





Multiple-Processor Scheduling – Load Balancing

- If SMP, need to keep all CPUs loaded for efficiency
- **Load balancing** attempts to keep workload evenly distributed
- **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs
- **Pull migration** – idle processors pulls waiting task from busy processor





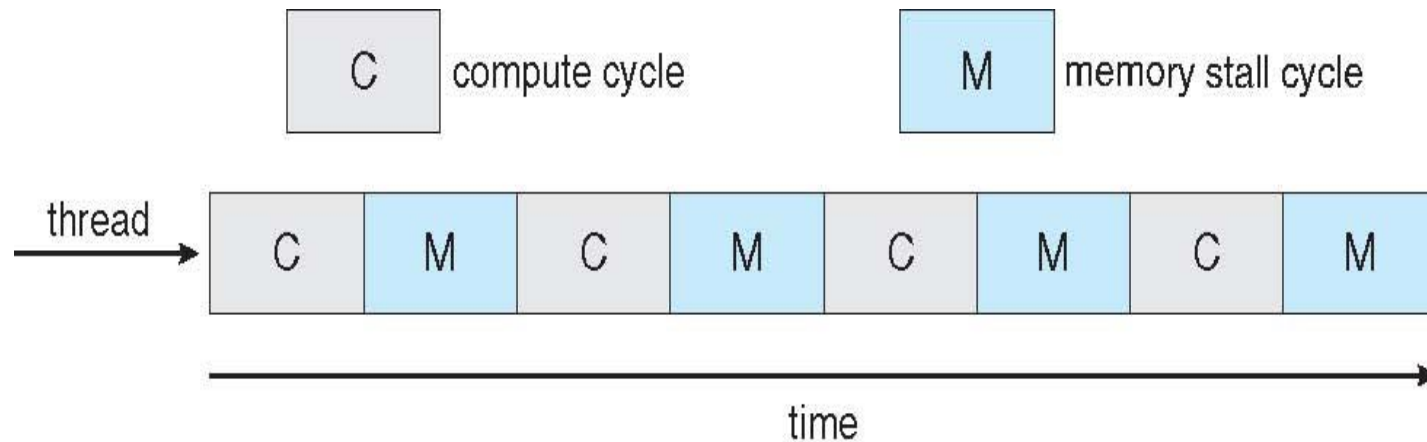
Multicore Processors

- Recent trend to place **multiple processor cores on same physical chip**
- Faster and consume less power
- Multiple threads per core also growing
 - Takes advantage of **memory stall** to make progress on another thread while memory retrieve happens(利用存储器的延迟, 当发生内存检索时另一个线程运行)





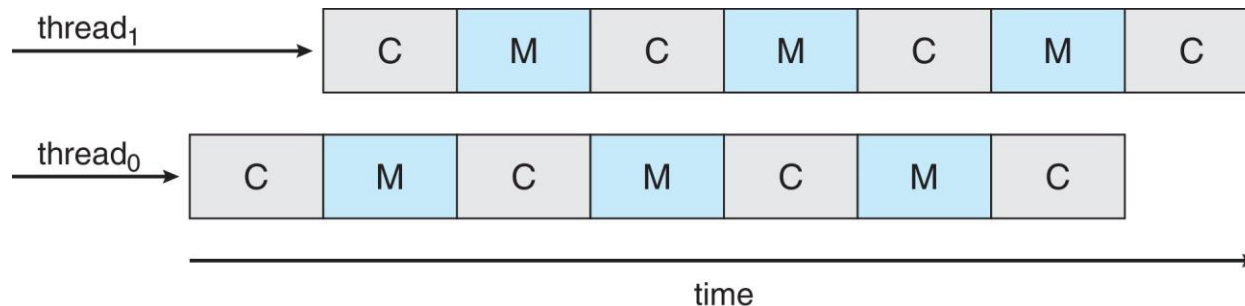
Multithreaded Multicore System





Multithreaded Multicore System

- Each core has > 1 hardware threads.
- If one thread has a memory stall, switch to another thread!
- Figure





Real time Scheduling

□ Real-time systems

Those systems in which the **correctness** of the system depends **not only** on the **logical result** of computation, **but also** on the **time** at which the results are produced.





Real time Scheduling

□ Real time computing is of two types

□ Hard real-time systems

- ▶ Requires to complete a critical task within a guaranteed amount of time
- ▶ **Special-purpose** systems
- ▶ failure to meet constraint is a **fatal fault**.

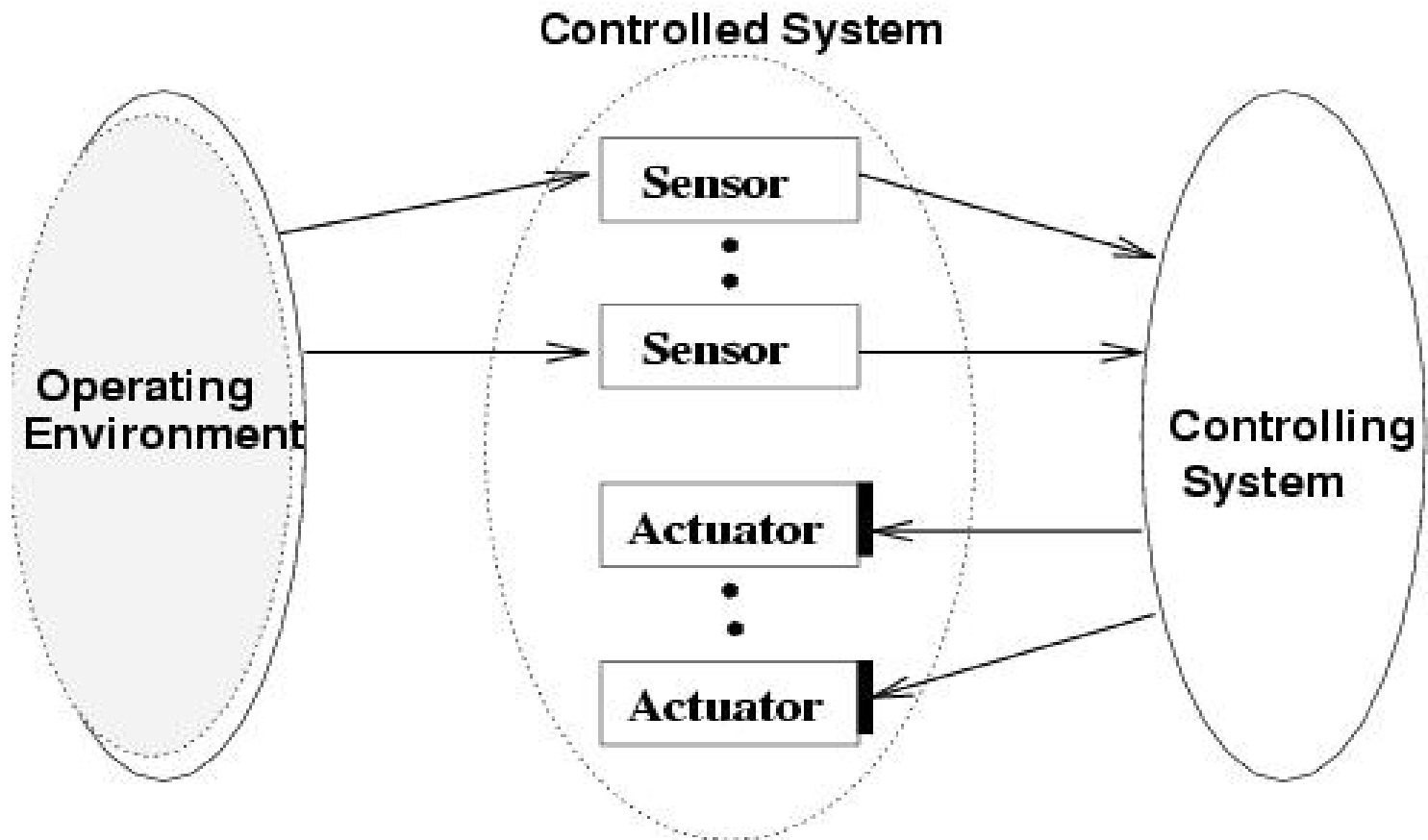
□ Soft real-time systems

- ▶ late completion is **undesirable** but generally not fatal.
- ▶ It requires that critical processes receive **priority** over less fortunate ones
- ▶ The result is a **general-purpose** system to support multimedia, high speed interactive graphics, and so on





A Sample Real Time System (1)





A Sample Real Time System (2)

- **Mission:** Reaching the destination safely.
- **Controlled System:** Car.
- **Operating environment:** Road conditions.
- **Controlling System**
 - *Human driver:* Sensors - Eyes and Ears of the driver.
 - *Computer:* Sensors - Cameras, Infrared receiver(红外探测), and Laser telemeter(激光测距).





A Sample Real Time System (3)

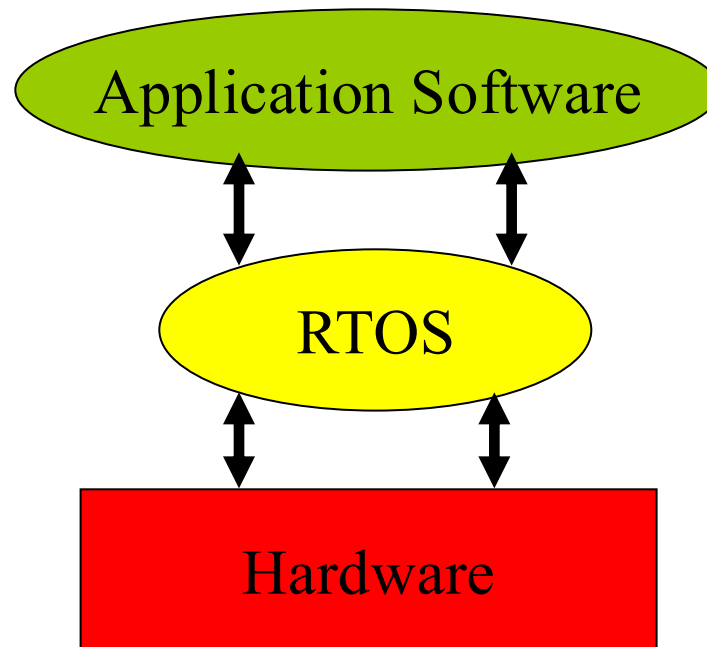
- **Controls:** Accelerator, Steering wheel, Break-pedal.
- **Actuators:** Wheels, Engines, and Brakes.
- **Critical tasks:** Steering and breaking
- **Non-critical tasks:** Turning on radio





RTOS Kernel

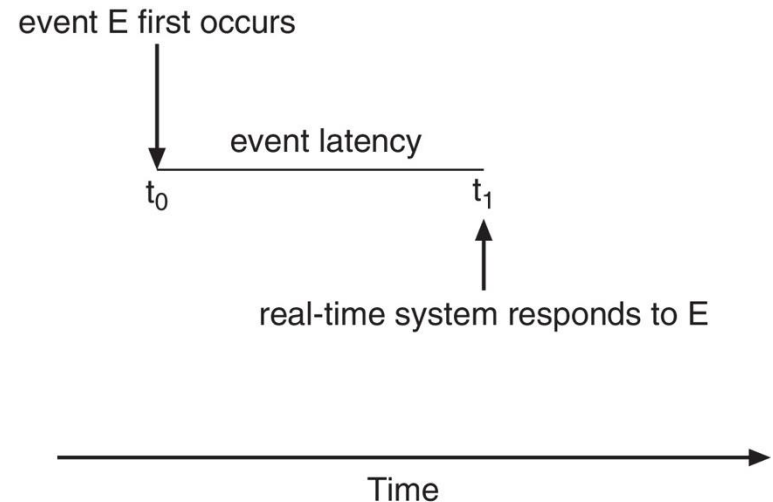
- RTOS Kernel provides an **Abstraction** layer that hides from application software the hardware details of the processor / set of processors upon which the application software shall run.





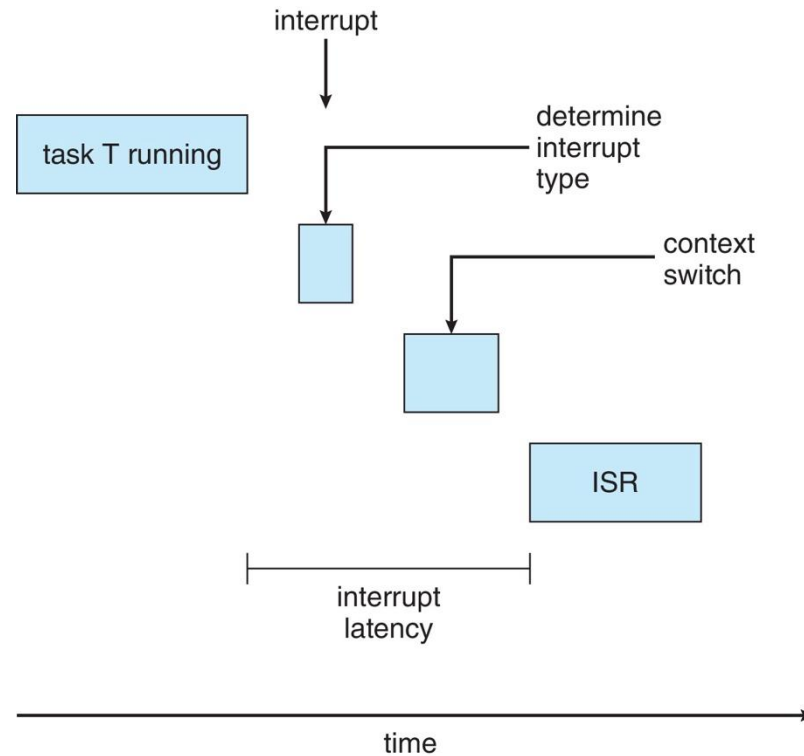
Real-Time CPU Scheduling

- Event latency – the amount of time that elapses from when an event **occurs** to when it is **served**.
- Two types of latencies affect performance
 1. **Interrupt latency** – time from arrival of interrupt to start of routine that services interrupt
 2. **Dispatch latency** – time for schedule to take current process off CPU and switch to another





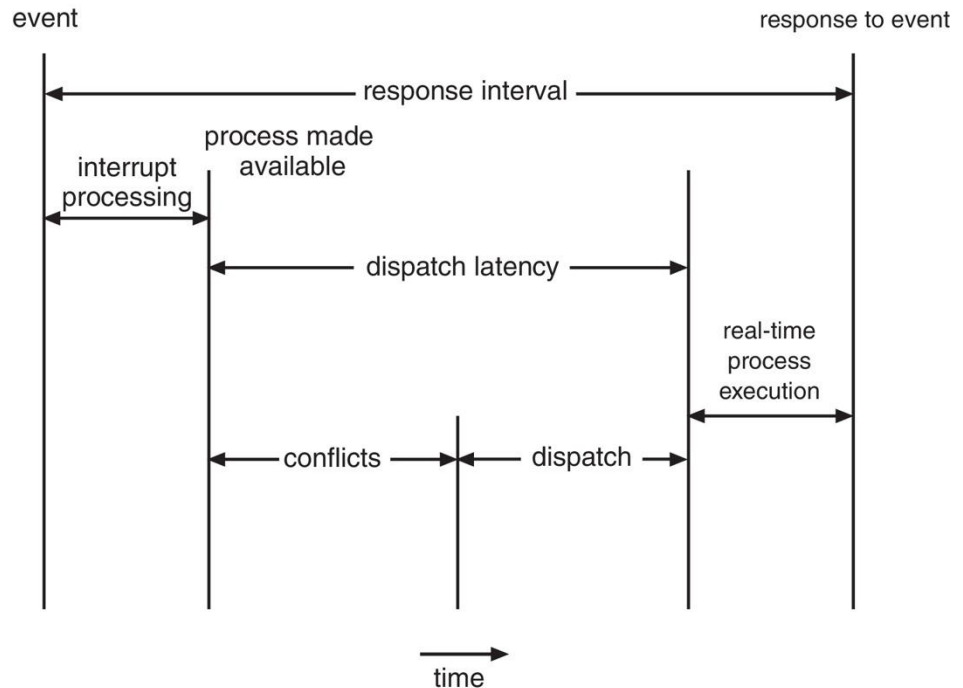
Interrupt Latency





Dispatch Latency

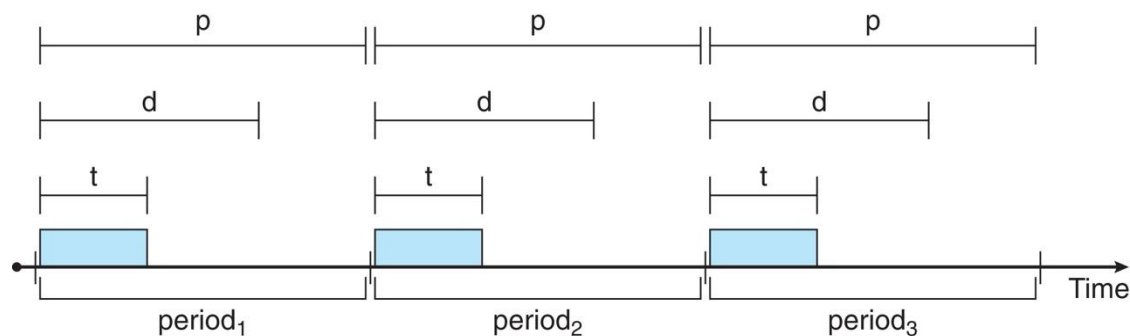
- Conflict phase of dispatch latency:
 1. **Preemption** of any process running in kernel mode
 2. **Release** by low-priority process of resources needed by high-priority processes





Priority-based Scheduling

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
 - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
- Processes have new characteristics: **periodic** ones require CPU at constant intervals
 - Has **processing** time t , **deadline** d , **period** p
 - $0 \leq t \leq d \leq p$
 - **Rate** of periodic task is $1/p$





Task Management

- Main service offered is Task Scheduling
 - controls the **execution** of application software tasks
 - can make them run in a very **timely and responsive fashion**.





Task Scheduling

- Non Real-time systems usually use Non-preemptive Scheduling
 - Once a task starts executing, it completes its full execution
- Most RTOS perform priority-based preemptive task scheduling.
- Basic rules for priority based preemptive task scheduling
 - The Highest Priority Task that is Ready to Run, will be the Task that Must be Running.





Real time Scheduling

- ❑ Typical RTOS based on **fixed-priority preemptive** scheduler
- ❑ Assign each process a priority
- ❑ At any time, scheduler runs highest priority process ready to run, **Higher Priority = Higher Need for Quick Response**
- ❑ Process runs to **completion unless preempted**
- ❑ Follows **nested preemption**





Real time Scheduling

- Typical real-time scheduling algorithm
 - EDF: Earliest Deadline First
 - ▶ **Dynamic** but more complex
 - ▶ Priority = **deadline**
 - RMS: rate-monotonic scheduling
 - ▶ Just one scalar priority related to the periodicity of the job
 - ▶ Priority = **1/rate**
 - ▶ **Static**





Rate-Monotonic Scheduling

- Common way to assign priorities
- Result from Liu & Layland, 1973 (JACM)
- Simple to understand and implement:

Processes with shorter period given higher priority

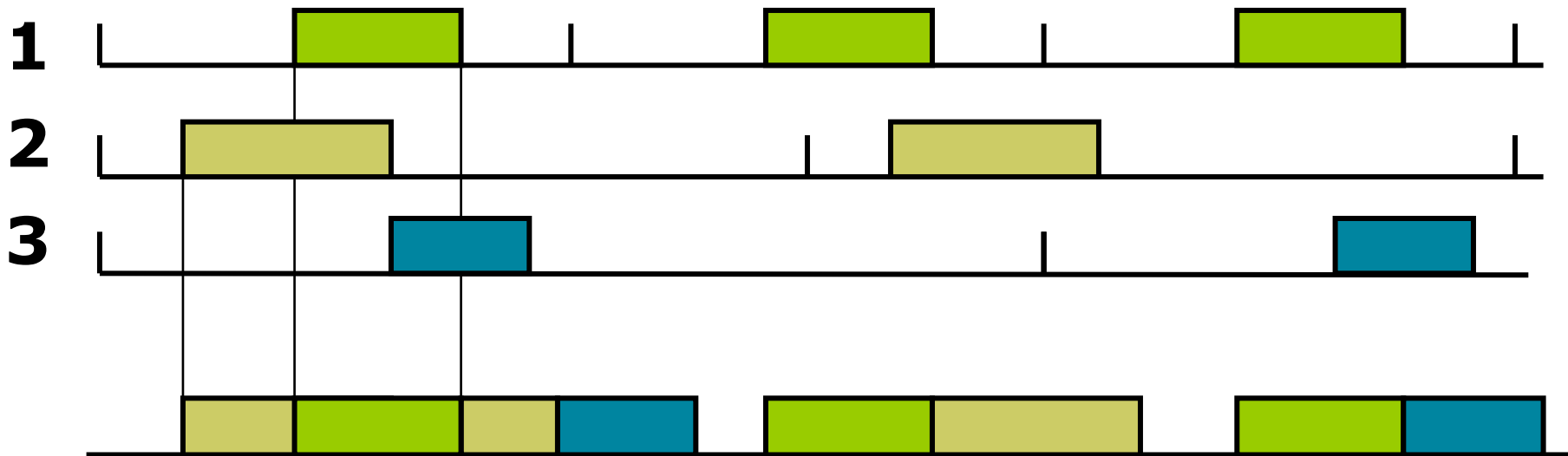
□ E.g.,	<u>Period</u>	<u>Priority</u>	
	10	1	(highest)
	12	2	
	15	3	
	20	4	(lowest)





Rate-Monotonic Scheduling

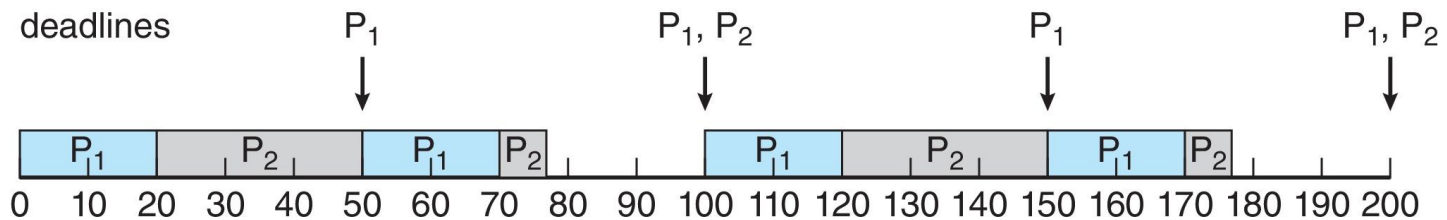
- Always run the highest-priority runnable process





Rate Monotonic Scheduling

- A priority is assigned based on the inverse of its period
- Shorter periods = higher priority;
- Longer periods = lower priority
- P_1 is assigned a higher priority than P_2 .



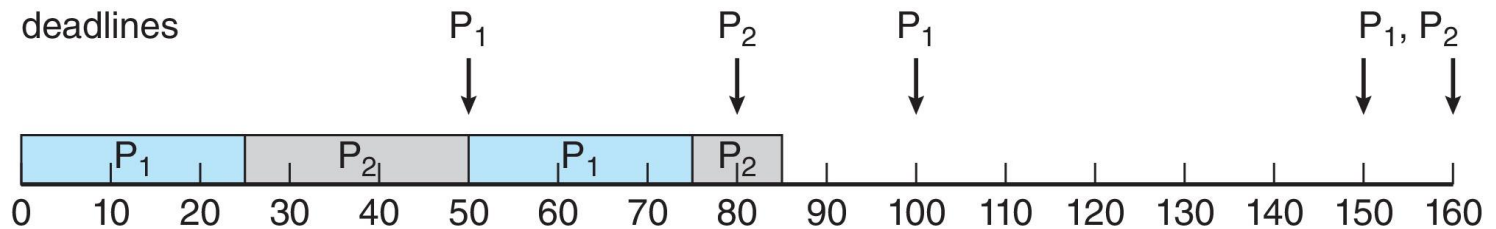
$$\begin{aligned} P_1 &= 50, & t_1 &= 20; & e_1 &= 20/50 = 0.4; \\ P_2 &= 100, & t_2 &= 35; & e_2 &= 35/100 = 0.35; \end{aligned}$$





Missed Deadlines with Rate Monotonic Scheduling

- Process P_2 misses finishing its deadline at time 80
- Figure



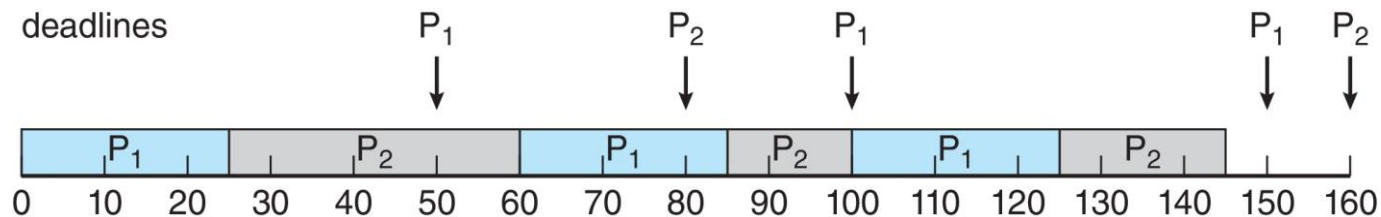
$P_1=50, \quad t_1=25;$
 $p_2=80, \quad t_2=35;$





Earliest Deadline First Scheduling (EDF)

- Priorities are assigned according to deadlines:
 - The earlier the deadline, the higher the priority
 - The later the deadline, the lower the priority
- Figure





Proportional Share Scheduling

- T shares are allocated among all processes in the system
- An application receives N shares where $N < T$
- This ensures each application will receive N / T of the total processor time





Earliest-Deadline-First (EDF) Scheduling

- Preemptive & dynamically assigns priorities according to deadline
 - Earlier deadline, higher priority
 - Later deadline, lower priority
- Highest priority process available gets CPU

triplet: (execution time, period, deadline)

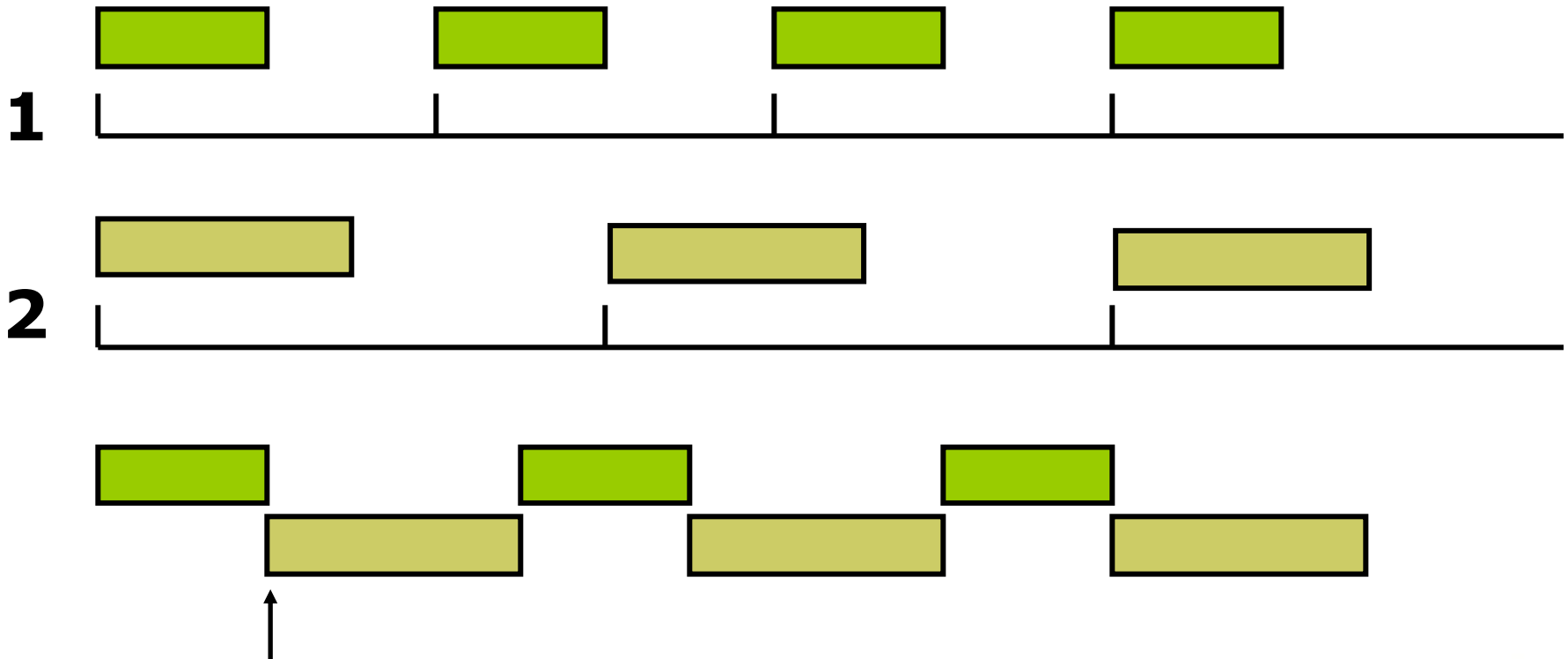




EDF Meeting a Deadline

triplet: (execution time, period, deadline)

□ $p1 = (10, 20, 20)$ $p2 = (15, 30, 30)$ utilization is 100%



P2 takes priority because its deadline is sooner





Dispatch Latency

- The conflict phase has two components:
 1. **Preemption** of any process running in the kernel ;
 2. Release by low-priority processes of **resources** needed by the high-priority process ;





Dispatch Latency

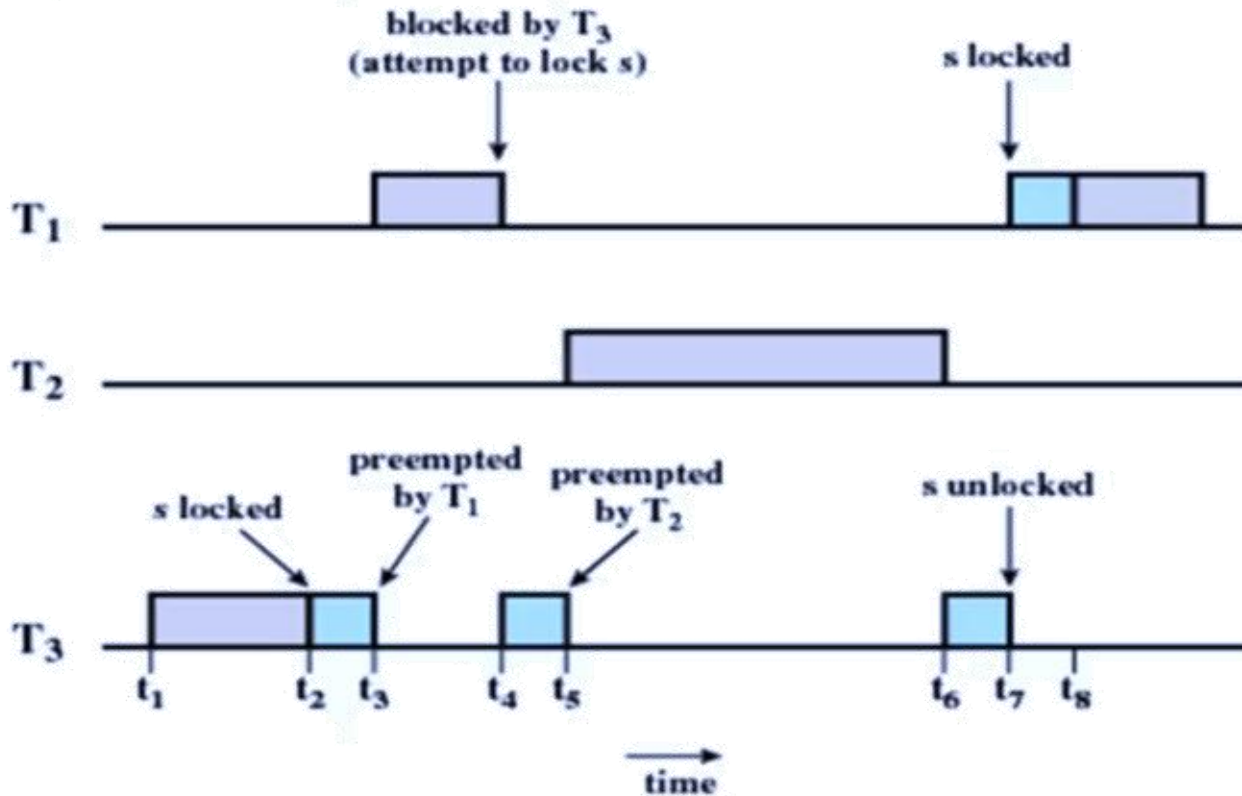
- What happens if the higher priority process needs to access kernel data that are currently being accessed by another, lower priority process?
- **Priority inversion:** the higher-priority process would be waiting for the lower priority process to finish.
- 该现象可能引起严重后果。例如，美国的火星探测器Mars Pathfinder就是因为优先级倒挂而出现故障。





Dispatch Latency

- 优先级反转的持续时间取决于其他不相关任务的不可预测的行为



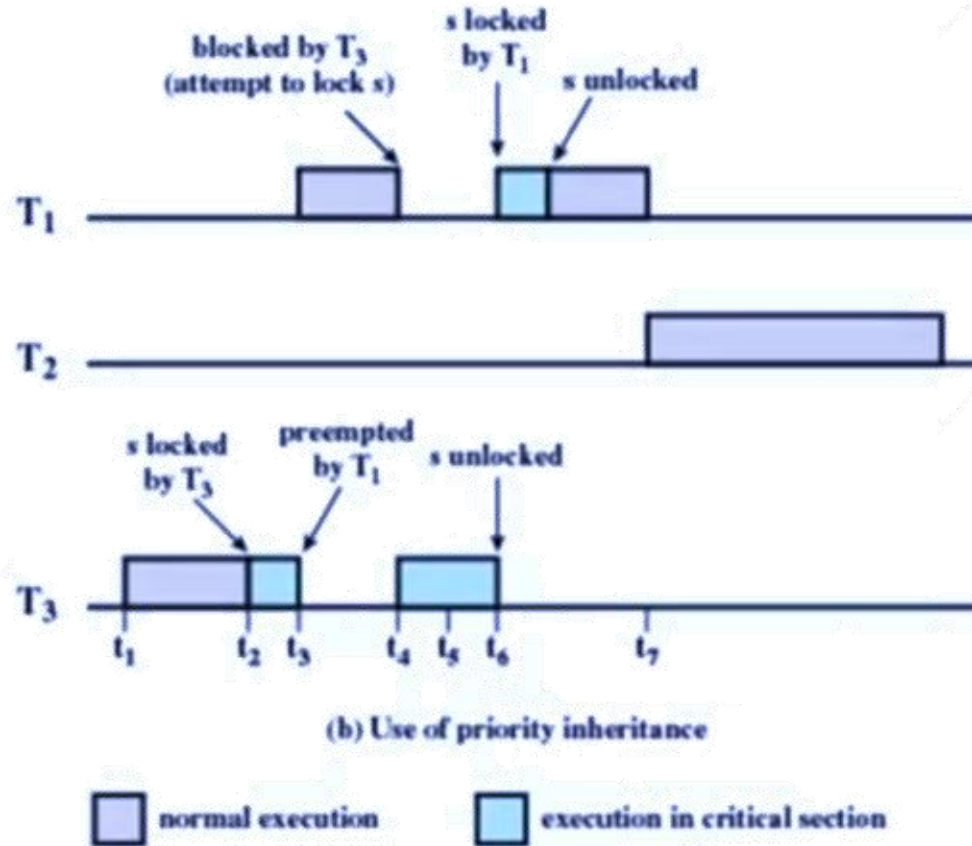
(a) Unbounded priority inversion





Dispatch Latency

- ◆ 低优先级任务继承高优先级任务的优先级依赖于他们共享的资源





Dispatch Latency

- Solution to Priority inversion:
 - **Priority-inheritance** protocol
 - The processes (accessing resources that the high-priority process needs) inherit the high priority until they are done with the resources in question.
 - When they are finished, their priority reverts to its original value.





Operating System Examples

- Solaris scheduling
- Windows XP scheduling
- Linux scheduling





Solaris scheduling

- ❑ Solaris uses priority-based thread scheduling.
- ❑ It defines 4 classes of scheduling , in order of priority:
 - ❑ Real time
 - ❑ System
 - ❑ Interactive
 - ❑ Time sharing
- ❑ Within each class there are different priorities and different scheduling algorithms.





Solaris scheduling

- The **default scheduling class** is **time sharing**.
- The scheduling policy for time sharing dynamically alters priorities and assigns time slices of different lengths using a **multilevel feedback queue**.
- The interactive class uses the same scheduling policy as the time sharing class, but it gives windowing applications a higher priority for better performance.
- Using system class to run kernel processes, such as the scheduler and paging daemon, **the priority of a system process does not change**.
- Threads in real time class are given **the highest priority**.



Solaris Dispatch Table for

**New priority of a thread that
has used its entire time
quantum**

**Priority of a thread
returning from sleeping**

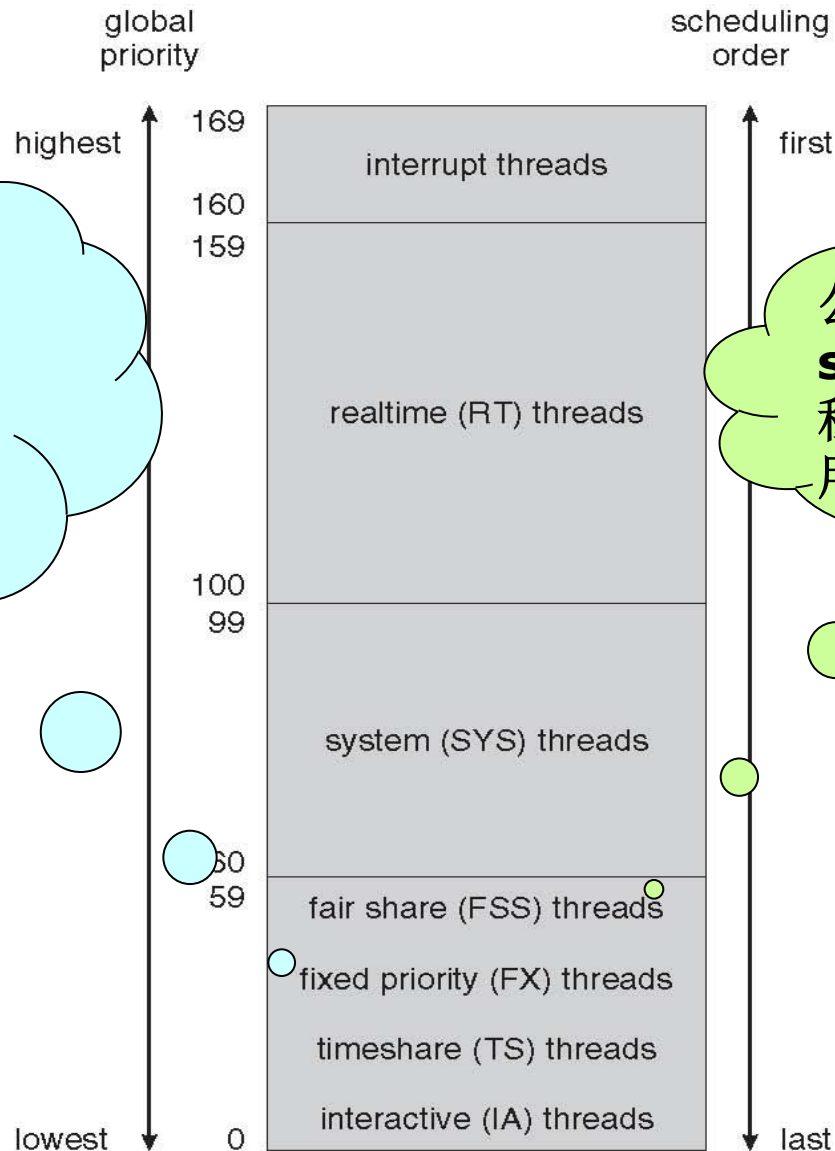
priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59





Solaris Scheduling

固定优先级(**fixed priority**)与分时的优先级在相同的范围,但线程优先级是不能动态改变的



公平共享(**fair share**)指一组进程共享**CPU**而不使用优先级调度.





Windows XP scheduling

- Priority-driven, preemptive scheduling system
- Thread runs for time amount of *quantum*
- Dispatcher routines triggered by the following events:
 - Thread becomes ready for execution
 - Thread leaves running state (quantum expires, wait state)
 - Thread's priority changes





Windows XP scheduling

- Multiple threads may be ready to run
- “Who gets to use the CPU?”
- From Windows API point of view:
 - Processes are given a priority class upon creation
 - ▶ Idle, Below normal, Normal, Above normal, High, Realtime
 - Threads have a relative priority within the class
 - ▶ Idle, Lowest, Below_Normal, Normal, Above_Normal, Highest, and Time_Critical





Windows XP Priorities

实时任务

非实时任务

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1





Windows XP Scheduling

- From the kernel's view:
 - Threads have priorities 0 through 31
 - **Threads are scheduled, not processes**
 - Process priority class is not used to make scheduling decisions





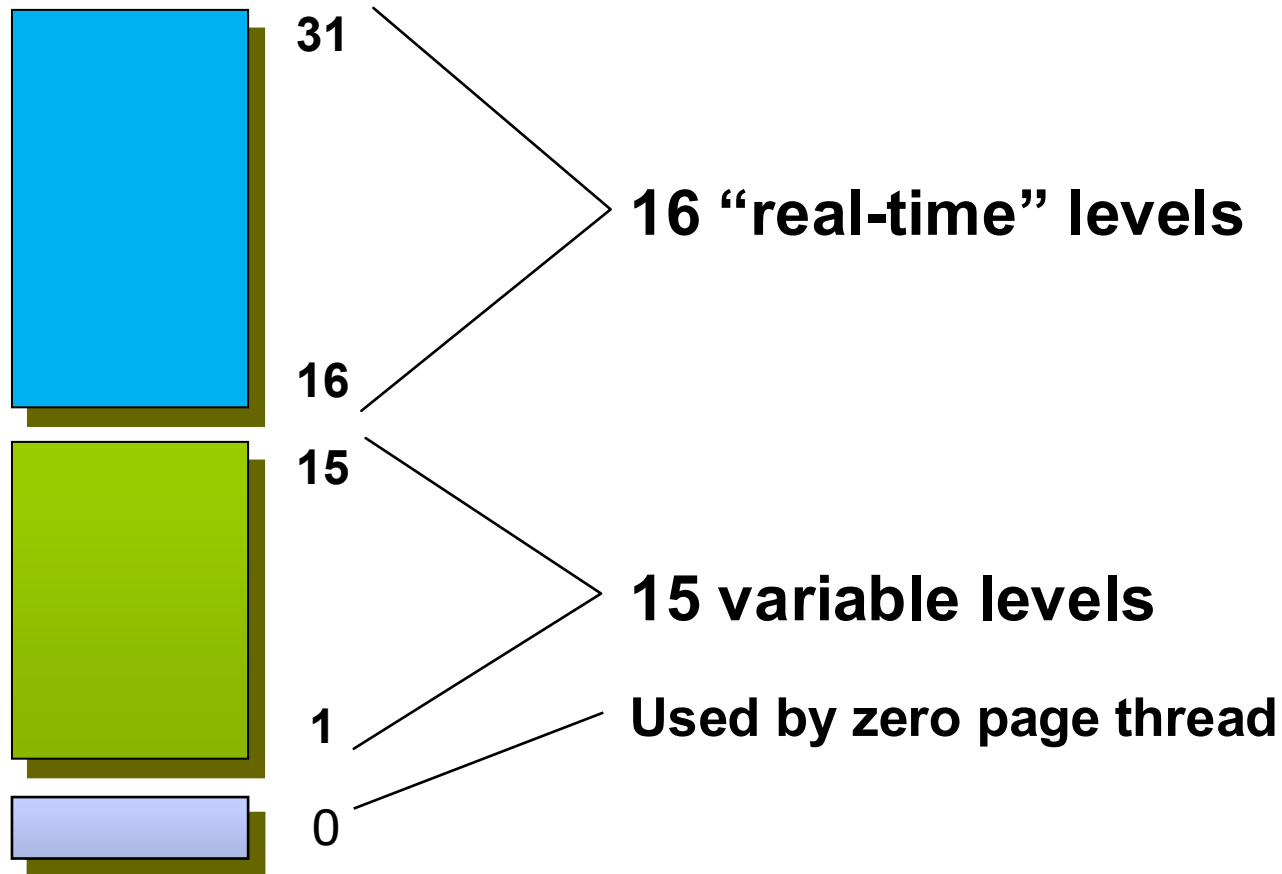
Windows XP Scheduling

- 32 priority levels
- Threads within same priority are scheduled following the Round-Robin policy
- Non-Realtime Priorities are adjusted dynamically
- Real-time priorities (i.e.; > 15) are assigned statically to threads





Kernel: Thread Priority Levels





Linux Scheduling

- The Linux scheduler is a preemptive, priority-based algorithm with two priority ranges: time-sharing and real-time
 - **Real-time** range from 0 to 99
 - **nice** value from 100 to 140
- These two ranges map into a global priority scheme whereby numerically **lower values indicate higher priority**





Priorities and Time-slice length

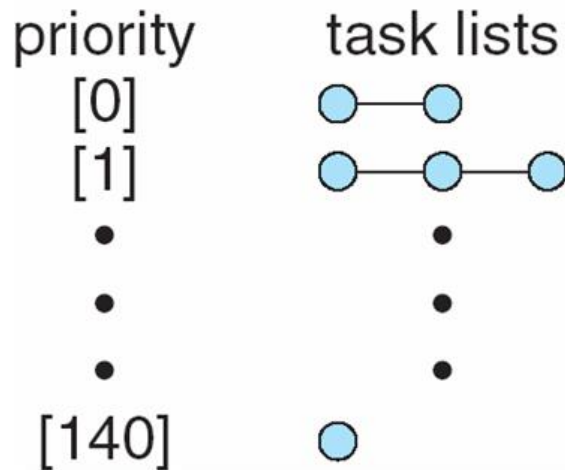
<u>numeric priority</u>	<u>relative priority</u>		<u>time quantum</u>
0	highest	real-time tasks	200 ms
•			
•			
•			
99			
100		other tasks	10 ms
•			
•			
•			
140	lowest		



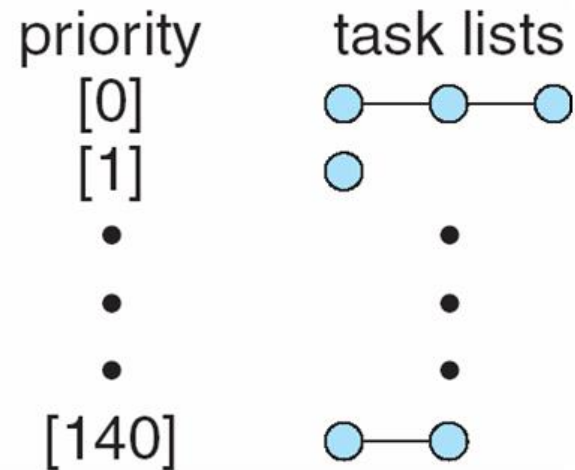


List of Tasks Indexed According to Priorities

**active
array**



**expired
array**





Linux Scheduling

- ❑ Linux supports SMP, each processor maintains its own runqueue and schedules itself independently.
- ❑ Each runqueue contains two priority arrays :
 - ❑ Active: contains all tasks with time remaining in their time slices
 - ❑ Expired: contains all expired tasks
 - ❑ Each of these arrays contains a list of tasks indexed according to priority .
- ❑ The scheduler chooses the task with the highest priority from the active array for execution on the CPU.
- ❑ When all tasks have exhausted their time slices, the priority arrays are exchanged.





Algorithm Evaluation

- How do we select a CPU scheduling algorithm for a particular system?
- The first problem is **defining the criteria** to be used in selecting an algorithm. According to system goal :
 - CPU utilization
 - Throughput
 - Average turnaround time
 - Response time
- Then select an algorithm
- And then evaluate the algorithm selected





Algorithm Evaluation

- Deterministic modeling
 - takes a particular predetermined workload and defines the performance of each algorithm for that workload
- Queueing models
- Simulations
- Implementation



End of Chapter 5

